

Import Statements

```
In [2]: import custom_cmap
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from astropy.io import fits
from astropy.visualization import PercentileInterval
from PIL import Image
from IPython.display import display
import matplotlib.pyplot as plt
import matplotlib as mpl
import pandas as pd
from moria import reduce
from pathlib import Path
from astropy.visualization import PercentileInterval

from astropy.io import fits
from astropy.visualization import LogStretch, ImageNormalize
import plotly.express as px
import numpy as np
from astropy.visualization import ImageNormalize, AsinhStretch, SqrtStretch, LogStretch, PowerStretch

plt.rcParams.update({'font.size':25})
plt.rc('text', usetex=True)
%config InlineBackend.figure_format = 'retina'
%load_ext autoreload
%autoreload 2
```

Understanding the main directory.

The data directory should be organized as follows. You can look at the sample data folder to understand the directory structure you need.

00.DATA
01.XYM
02.CMD
03.LOC_TRANS
04.PSF_EXTRACT
05.COORD_TRANS(OPTIONAL)
06.FIT
07.CALIBRATION

All the necessary scripts are included in the sample data directory. The simplest way to run MORIA on your target is to copy the entirety of the "data" folder here, put your exposures in "data/00.DATA", then beginning with the demo.

The target star is modeled using a one, two or three-star (optional) PSF model to account for the source star, lens star and possible nearby companions. Pixels surrounding the target are fit using these different PSF models using the Markov Chain Monte Carlo (MCMC) algorithm that solves for stellar positions and flux fractions.

After you finish running this notebook, you will be able to use the "_flc" exposures to generate an output image stack in the F814W and F606W HST filters.

NOTE: `chmod +x program.src` is a useful command to use whenever permissions are denied for a script.

```
In [3]: #This is the directory where you are processing your data. This does not point at MORIA.  
directory = os.getcwd()
```

Step 0

We assume you ran the `output_stacks.ipynbm`, `cmd_diagram.ipynb`, `creating_psf.ipynb` (in that order) notebooks correctly

Step 1

The goal of this penultimate step is to fit the pixels of the target star with the PSF to determine what the best-fit 2 or 3-star model is. We also have the option of applying the lens-source separation constraint from the Keck analysis. We will begin by doing this for the F814W filter.

We will now design an input file that helps us run the 1star and 2star fit for F814W. [TBD Describe the inputs more]

```
In [18]: reduce.hst_fit_dataprep_twostar(directory) # Input for 2star-fit F814W filter
```

```
In [8]: reduce.hst_fit_dataprep_onestar(directory) # Input for 1star-fit F814W filter
```

```
In [26]: reduce.hst_fit_final_F814W(directory) # Run the tri fitting
```

Step 2

Look at the Residuals for the 1star-fit and 2star-fit. These fitting results are kept in 06.FIT/F814W for the F814W filter and 06.FIT/F606W for the F606W filter.

The co-ordinate system we show below is the 'outputq' frame where the target is at (0, 0).

The pixel coordinates are scaled accordingly. Given the 1star-fit and 2star-fit, if you opt for a 3star-fit, you can approximately make your IN.* file using these fits.

```
In [43]: fit_file = Path(directory).resolve()/f"06.FIT/F814W/1star-fit/uvp2tri_scon_fsky_I_KeckN0con.06.pix_show.fi
fname = Path(directory).resolve()/f"06.FIT/F814W/1star-fit/expanded_mcmc.txt"

xtarg = 3001
ytarg = 1001

colname1 = ['X1_CENTER', 'Y1_CENTER']
chains1 = pd.read_table(fname, usecols=[0,1], sep=r'\s+', skiprows=1, names=colname1).head(100)

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()
interval = PercentileInterval(98)
scaled = interval(data)

# image dimensions
ny, nx = scaled.shape
```

```

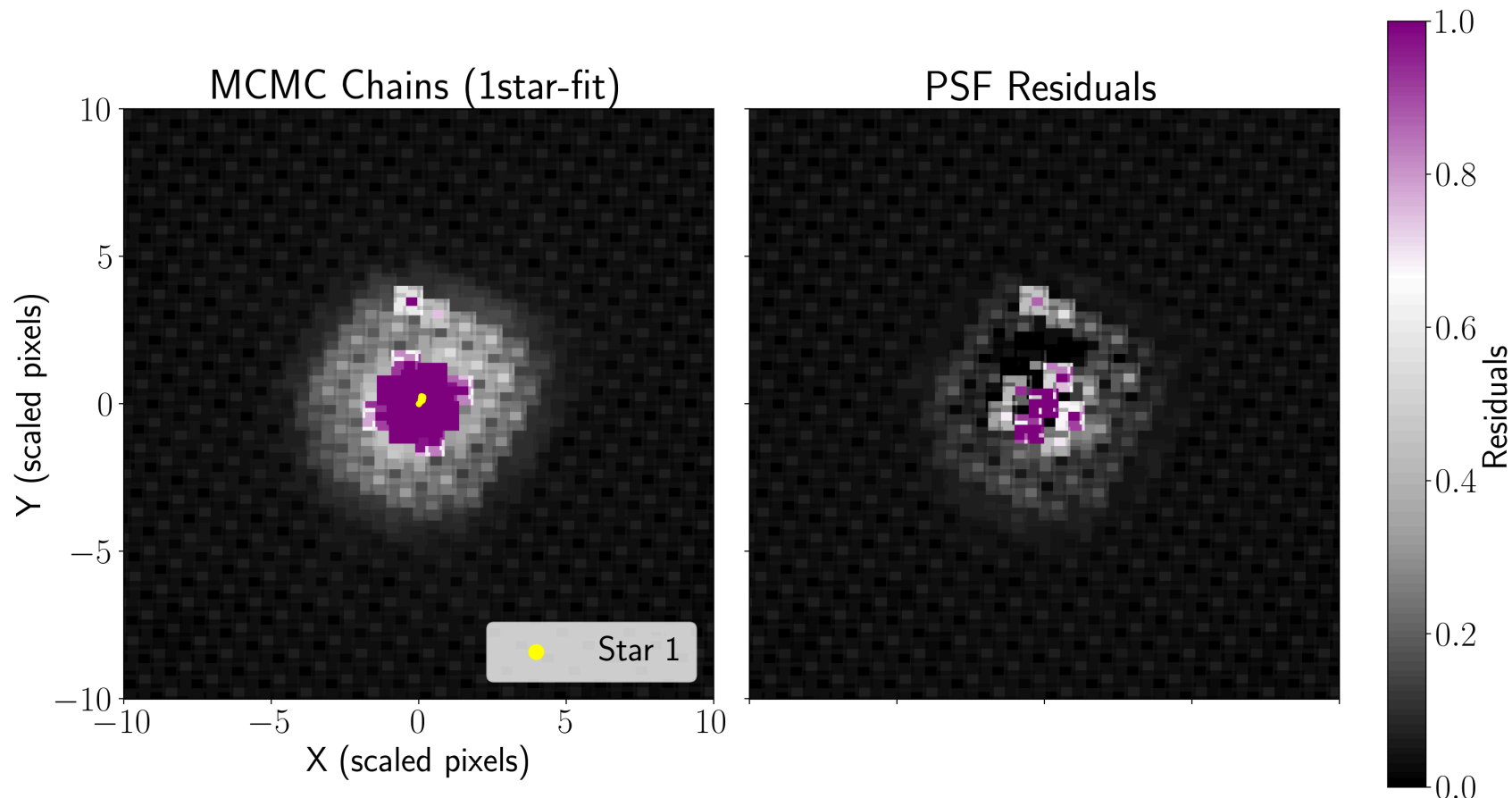
# scaled physical coordinates
x_extent = ((0 - xtarg)/100, (nx - xtarg)/100)
y_extent = ((0 - ytarg)/100, (ny - ytarg)/100)

fig, ax = plt.subplots(1,2, figsize=(15, 8), constrained_layout=True)
im = ax[0].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=
ax[0].scatter(chains1['X1_CENTER'], chains1['Y1_CENTER'], s=12, color='yellow', label='Star 1')
ax[0].legend(loc='lower right', markerscale=3)
cbar = fig.colorbar(im, ax=[ax[0], ax[1]], fraction=0.046, pad=0.04)
cbar.set_label('Residuals')
ax[0].set_title("MCMC Chains (1star-fit)")
ax[0].set_xlabel("X (scaled pixels)")
ax[0].set_ylabel("Y (scaled pixels)")
ax[0].set_xlim(-10, 10)

ax[1].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=cus
ax[1].set_xlim(-30, -10)
ax[1].set_title("PSF Residuals")
ax[1].set_xticklabels([])
ax[1].set_yticklabels([])

plt.show()

```



The panel on the left is the target in the HST images. It is not the PSF!

Using the PSF we generated in "creating_psf.ipynb", we try fitting a single PSF on the target. The right panel shows the result of subtracting the PSF from the target. For the 1star PSF fit, we clearly still see big residuals. Ideally, if the residual is smooth on the right panel, the PSF was "well-subtracted" from the target.

Therefore, looking at the dipole structure for the 1star PSF fitting result above, it is clear that running a 2star PSF fitting (or even a 3star PSF fitting) is a good idea!

```
In [35]: fit_file = Path(directory).resolve()/f"06.FIT/F814W/2star-fit/uvp2tri_scon_fsky_I_KeckN0con.06.pix_show.fi
         fname = Path(directory).resolve()/f"06.FIT/F814W/2star-fit/expanded_mcmc.txt"
```

```

xtarg = 3001
ytarg = 1001

colname1 = ['X1_CENTER', 'Y1_CENTER']
colname2 = ['X2_CENTER', 'Y2_CENTER']
chains1 = pd.read_table(fname, usecols=[0,1], sep=r'\s+', skiprows=1, names=colname1).head(100)
chains2 = pd.read_table(fname, usecols=[2,3], sep=r'\s+', skiprows=1, names=colname2).head(100)

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()

interval = PercentileInterval(98)
scaled = interval(data)

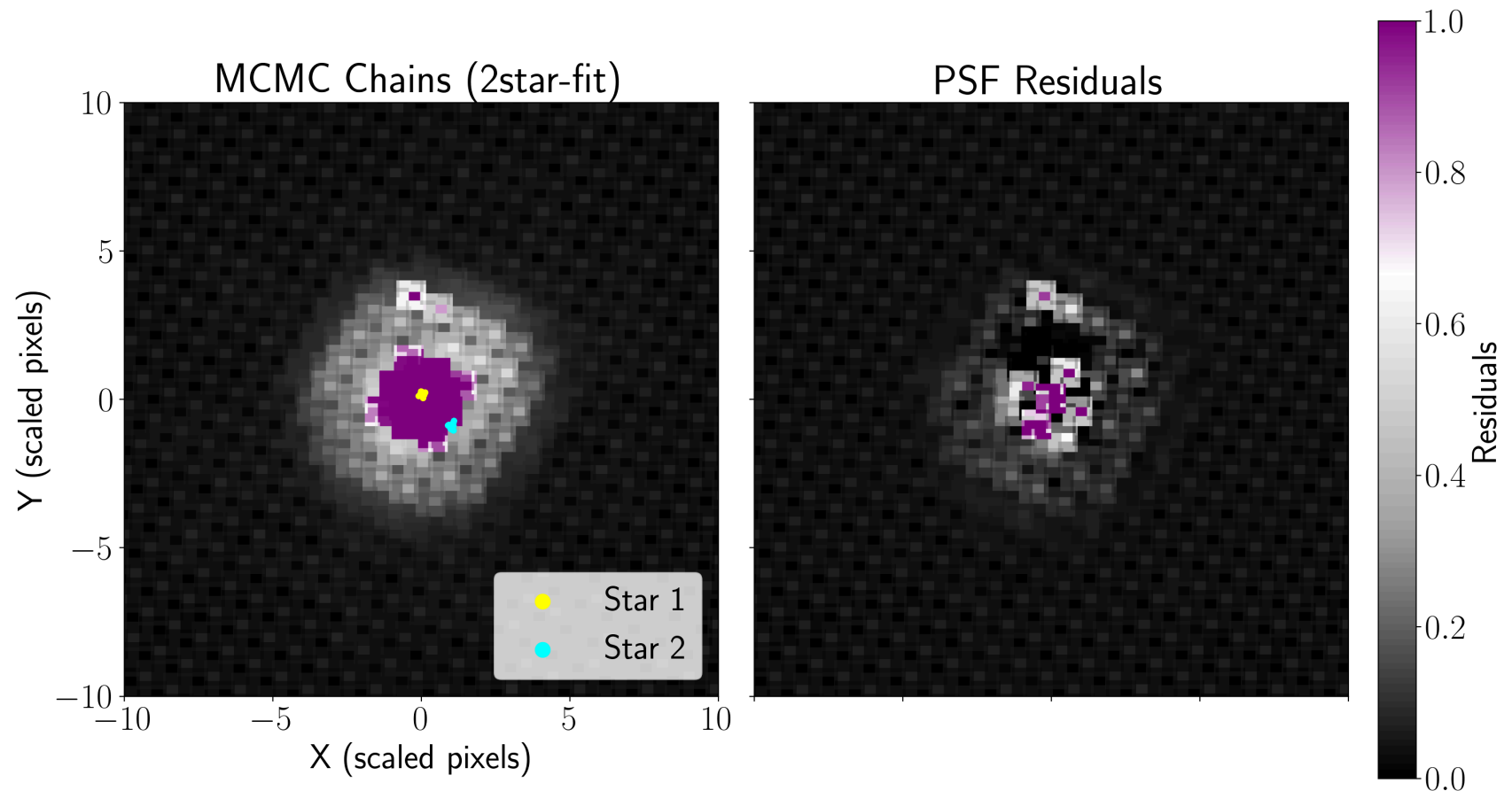
# image dimensions
ny, nx = scaled.shape

# scaled physical coordinates
x_extent = ((0 - xtarg)/100, (nx - xtarg)/100)
y_extent = ((0 - ytarg)/100, (ny - ytarg)/100)
fig, ax = plt.subplots(1,2, figsize=(15, 8), constrained_layout=True)
im = ax[0].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=
ax[0].scatter(chains1['X1_CENTER'], chains1['Y1_CENTER'], s=12, color='yellow', label='Star 1')
ax[0].scatter(chains2['X2_CENTER'], chains2['Y2_CENTER'], s=12, color='cyan', label='Star 2')
ax[0].legend(loc='lower right', markerscale=3)
cbar = fig.colorbar(im, ax=ax[1])
cbar.set_label('Residuals')
ax[0].set_title("MCMC Chains (2star-fit)")
ax[0].set_xlabel("X (scaled pixels)")
ax[0].set_ylabel("Y (scaled pixels)")
ax[0].set_xlim(-10, 10)

ax[1].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=cus
ax[1].set_xlim(-30, -10)
ax[1].set_title("PSF Residuals")
ax[1].set_xticklabels([])
ax[1].set_yticklabels([])

plt.show()

```



If you are not satisfied with these residuals, repeat the step. If you think a 3star fit might be a better option, continue below

Step 3

If you are still unhappy with the 2 star residuals, consider running a 3star-fit. The 3star-fit does not run by default in MORIA.

```
In [71]: reduce.hst_fit_dataprep_threestar(directory) # Input for 3star-fit F814W filter
```

```
In [53]: reduce.hst_fit_dataprep_threestar(directory) # Input for 3star-fit F814W filter
```

```
In [72]: reduce.tri_fit_F814W_opt(directory)
```

Step 4

We will look at the 3star-fit PSF residuals.

```
In [91]: chains4
```

```
Out [91]:
```

	X3_CENTER	Y3_CENTER
0	0.5649	0.305536019.472320347.039711004.6692
1	0.6314	0.285536311.975922928.834110365.9798
2	0.5459	0.269437826.108520649.272610191.8667
3	0.5046	0.281339275.561519818.841111047.4299
4	0.5046	0.281339275.561519818.841111047.4299
...	...	...
95	0.3736	0.212542909.975116031.5958
96	0.3297	0.188342681.670114070.8662
97	0.3344	0.241443671.229414601.912310542.671518526.6456
98	0.2984	0.237444265.906913209.389310506.513020550.0046
99	0.2984	0.237444265.906913209.389310506.513020550.0046

100 rows × 2 columns

```
In [11]: fit_file = Path(directory).resolve()/f"06.FIT/F814W/3star-fit/uvp2tri_scon_fsky_I_KeckN0con.06.pix_show.fi
fname = Path(directory).resolve()/f"06.FIT/F814W/3star-fit/expanded_mcmc.txt"

xtarg = 3001
ytarg = 1001
```



```

colname1 = ['X1_CENTER', 'Y1_CENTER']
colname2 = ['X2_CENTER', 'Y2_CENTER']
colname3 = ['X3_CENTER', 'Y3_CENTER']

chains1 = pd.read_table(fname, usecols=[0,1], sep=r'\s+', skiprows=1, names=colname1).head(100)
chains2 = pd.read_table(fname, usecols=[2,3], sep=r'\s+', skiprows=1, names=colname2).head(100)
chains3 = pd.read_table(fname, usecols=[4,5], sep=r'\s+', skiprows=1, names=colname3).head(100)

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()

interval = PercentileInterval(98)
scaled = interval(data)

# image dimensions
ny, nx = scaled.shape
print(ny, nx)
# scaled physical coordinates
x_extent = ((0 - xtarg)/100, (nx - xtarg)/100)
y_extent = ((0 - ytarg)/100, (ny - ytarg)/100)

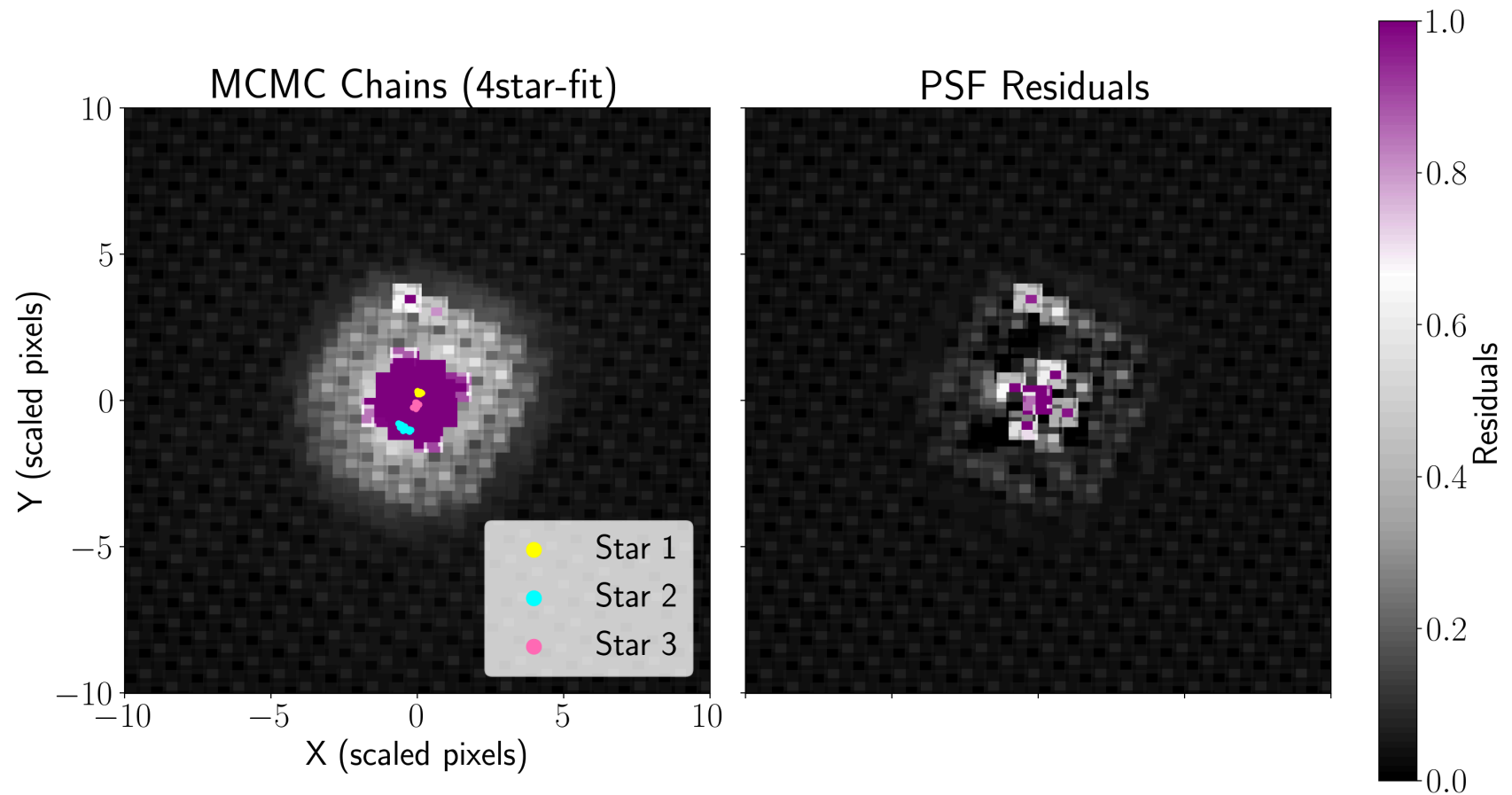
fig, ax = plt.subplots(1,2, figsize=(15, 8), constrained_layout=True)
im = ax[0].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=
ax[0].scatter(chains1['X1_CENTER'], chains1['Y1_CENTER'], s=12, color='yellow', label='Star 1')
ax[0].scatter(chains2['X2_CENTER'], chains2['Y2_CENTER'], s=12, color='cyan', label='Star 2')
ax[0].scatter(chains3['X3_CENTER'], chains3['Y3_CENTER'], s=12, color='hotpink', label='Star 3')

ax[0].legend(loc='lower right', markerscale=3)
cbar = fig.colorbar(im, ax=[ax[0], ax[1]], fraction=0.046, pad=0.04)
cbar.set_label('Residuals')
ax[0].set_title("MCMC Chains (4star-fit)")
ax[0].set_xlabel("X (scaled pixels)")
ax[0].set_ylabel("Y (scaled pixels)")
ax[0].set_xlim(-10, 10)

ax[1].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=cus
ax[1].set_xlim(-30, -10)
ax[1].set_title("PSF Residuals")

```

```
ax[1].set_xticklabels([])  
ax[1].set_yticklabels([])  
  
plt.show()
```



The 3Star PSF fitting still leaves room for error. You can re-run the cells above to try a new different set of parameters.

Step 5

Repeat the tri fitting process for the 606W filter by running the cells below.

```
In [14]: reduce.hst_fit_dataprep_twostar(directory, f='F606W') # Input for 2star-fit F606W filter

In [15]: reduce.hst_fit_dataprep_onestar(directory, f='F606W') # Input for 1star-fit F606W filter

In [17]: reduce.hst_fit_dataprep_threestar(directory, f = 'F606W') # Input for 3star-fit F606W filter

In [18]: reduce.hst_fit_final_F606W(directory) # Run the tri fitting

In [23]: reduce.tri_fit_F606W_opt(directory)
```

Step 6

Look at the Residuals for the 1star-fit, 2star-fit and 3star-fit for the F606W filter.

```
In [24]: fit_file = Path(directory).resolve()/f"06.FIT/F606W/1star-fit/uvp2tri_scon_fsky_V_KeckN0con.06.pix_show.fi
fname = Path(directory).resolve()/f"06.FIT/F606W/1star-fit/expanded_mcmc.txt"

xtarg = 3001
ytarg = 1001

colname1 = ['X1_CENTER', 'Y1_CENTER']
chains1 = pd.read_table(fname, usecols=[0,1], sep=r'\s+', skiprows=1, names=colname1).head(100)

chains1['X1_CENTER'] = (chains1['X1_CENTER'])
chains1['Y1_CENTER'] = (chains1['Y1_CENTER'])

hdl = fits.open(fit_file)
data = hdl[-1].data
hdl.close()

interval = PercentileInterval(90)
scaled = interval(data)
```

```

# image dimensions
ny, nx = scaled.shape

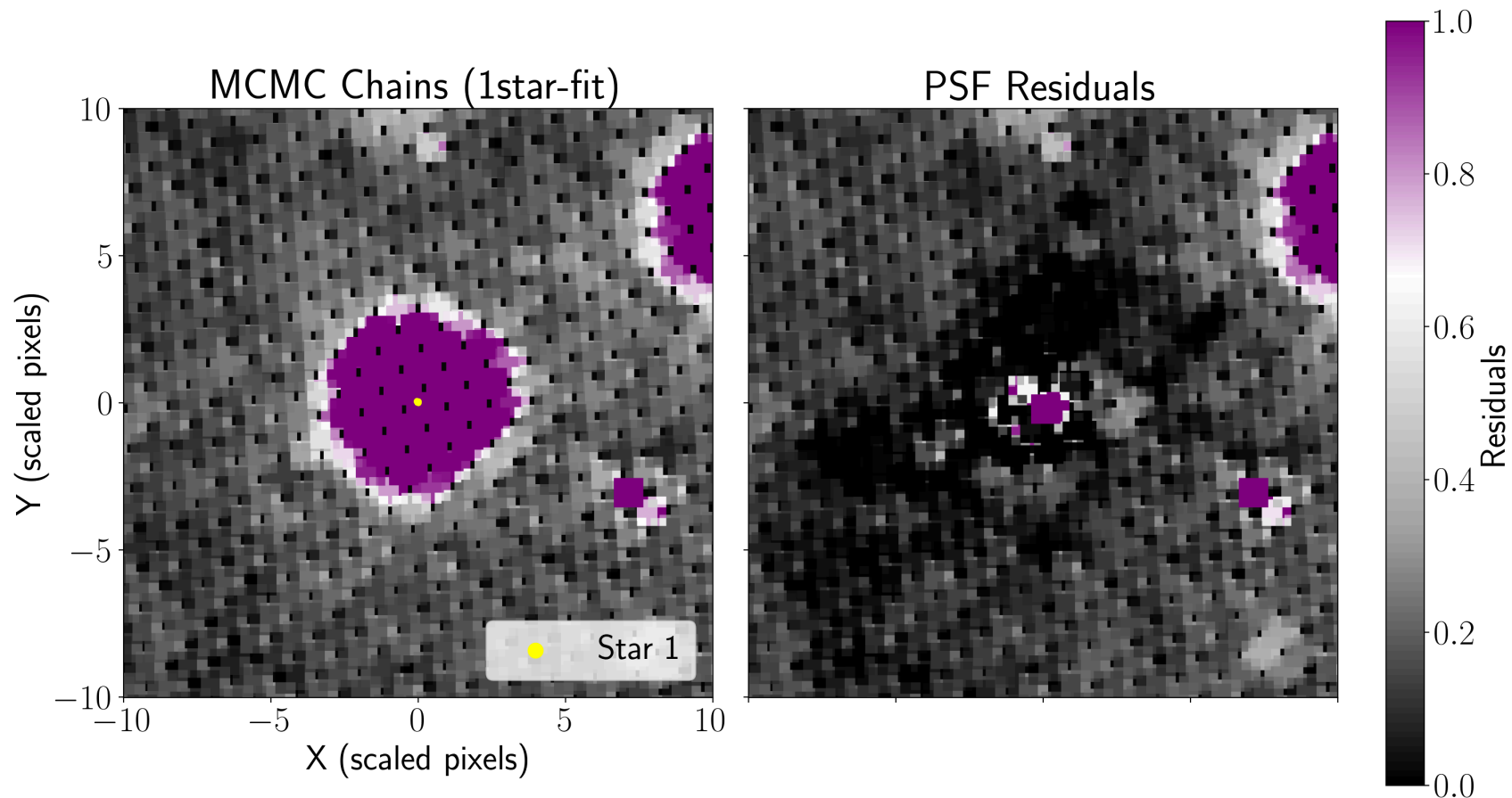
# scaled physical coordinates
x_extent = ((0 - xtarg)/100, (nx - xtarg)/100)
y_extent = ((0 - ytarg)/100, (ny - ytarg)/100)

fig, ax = plt.subplots(1,2, figsize=(15, 8), constrained_layout=True)
im = ax[0].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=
ax[0].scatter(chains1['X1_CENTER'], chains1['Y1_CENTER'], s=12, color='yellow', label='Star 1')
ax[0].legend(loc='lower right', markerscale=3)
cbar = fig.colorbar(im, ax=[ax[0], ax[1]], fraction=0.046, pad=0.04)
cbar.set_label('Residuals')
ax[0].set_title("MCMC Chains (1star-fit)")
ax[0].set_xlabel("X (scaled pixels)")
ax[0].set_ylabel("Y (scaled pixels)")
ax[0].set_xlim(-10, 10)

ax[1].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=cus
ax[1].set_xlim(-30, -10)
ax[1].set_title("PSF Residuals")
ax[1].set_xticklabels([])
ax[1].set_yticklabels([])

plt.show()

```



```
In [25]: fit_file = Path(directory).resolve()/f"06.FIT/F606W/2star-fit/uvp2tri_scon_fsky_V_KeckN0con.06.pix_show.fi
fname = Path(directory).resolve()/f"06.FIT/F606W/2star-fit/expanded_mcmc.txt"

xtarg = 3001
ytarg = 1001

colname1 = ['X1_CENTER', 'Y1_CENTER']
colname2 = ['X2_CENTER', 'Y2_CENTER']
chains1 = pd.read_table(fname, usecols=[0,1], sep=r'\s+', skiprows=1, names=colname1).head(100)
chains2 = pd.read_table(fname, usecols=[2,3], sep=r'\s+', skiprows=1, names=colname2).head(100)

chains1['X1_CENTER'] = (chains1['X1_CENTER'])
chains1['Y1_CENTER'] = (chains1['Y1_CENTER'])
```

```

chains2['X2_CENTER'] = (chains2['X2_CENTER'])
chains2['Y2_CENTER'] = (chains2['Y2_CENTER'])

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()

interval = PercentileInterval(90)
scaled = interval(data)

# image dimensions
ny, nx = scaled.shape

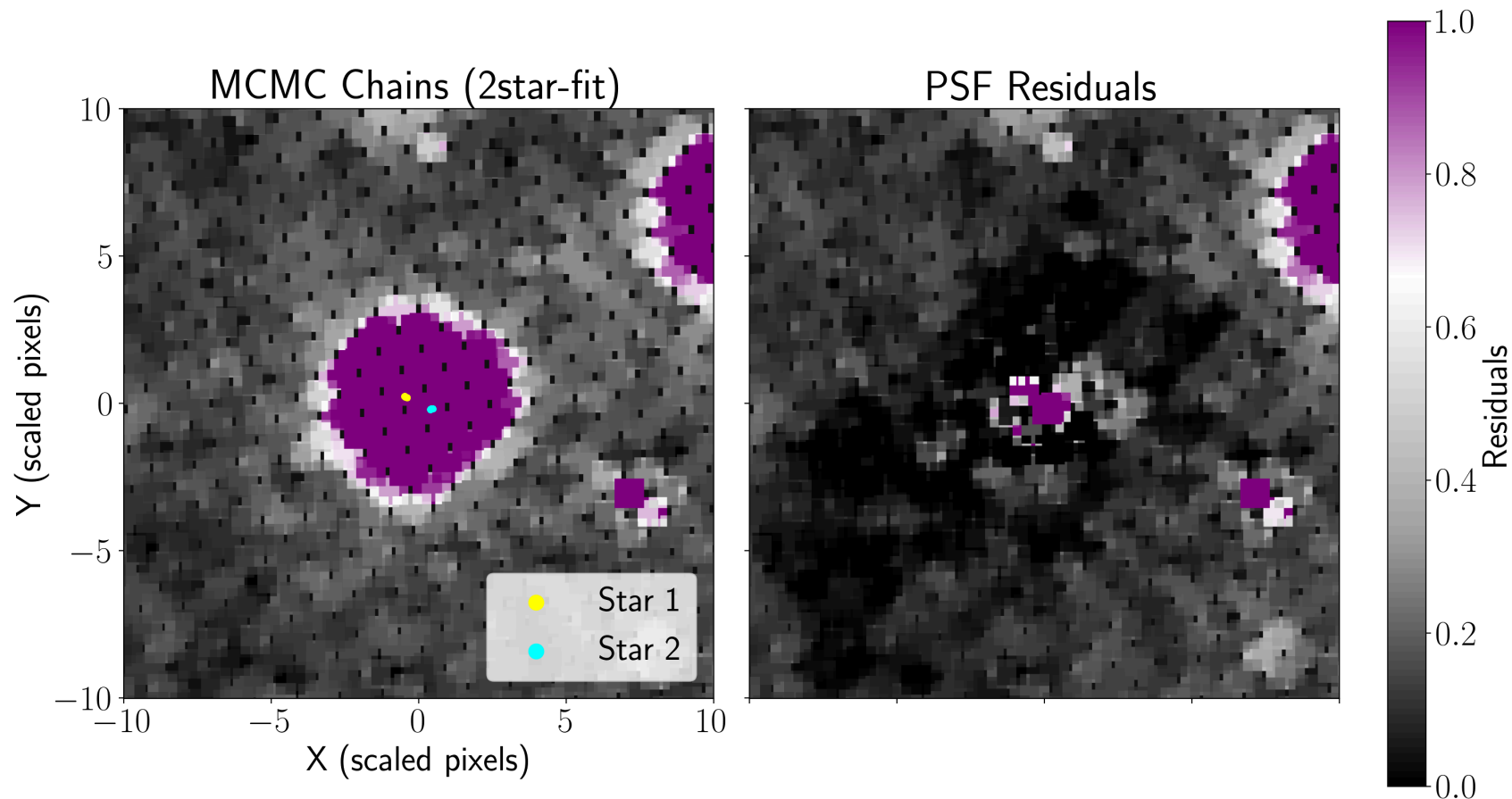
# scaled physical coordinates
x_extent = ((0 - xtarg)/100, (nx - xtarg)/100)
y_extent = ((0 - ytarg)/100, (ny - ytarg)/100)

fig, ax = plt.subplots(1,2, figsize=(15, 8), constrained_layout=True)
im = ax[0].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=
ax[0].scatter(chains1['X1_CENTER'], chains1['Y1_CENTER'], s=12, color='yellow', label='Star 1')
ax[0].scatter(chains2['X2_CENTER'], chains2['Y2_CENTER'], s=12, color='cyan', label='Star 2')
ax[0].legend(loc='lower right', markerscale=3)
cbar = fig.colorbar(im, ax=[ax[0], ax[1]], fraction=0.046, pad=0.04)
cbar.set_label('Residuals')
ax[0].set_title("MCMC Chains (2star-fit)")
ax[0].set_xlabel("X (scaled pixels)")
ax[0].set_ylabel("Y (scaled pixels)")
ax[0].set_xlim(-10, 10)

ax[1].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=cus
ax[1].set_xlim(-30, -10)
ax[1].set_title("PSF Residuals")
ax[1].set_xticklabels([])
ax[1].set_yticklabels([])

plt.show()

```



```
In [ ]: fit_file = Path(directory).resolve()/f"06.FIT/F606W/3star-fit/uvp2tri_scon_fsky_V_KeckN0con.06.pix_show.fi
fname = Path(directory).resolve()/f"06.FIT/F606W/3star-fit/expanded_mcmc.txt"

xtarg = 3001
ytarg = 1001

colname1 = ['X1_CENTER', 'Y1_CENTER']
colname2 = ['X2_CENTER', 'Y2_CENTER']
colname3 = ['X3_CENTER', 'Y3_CENTER']

chains1 = pd.read_table(fname, usecols=[0,1], sep=r'\s+', skiprows=1, names=colname1).head(100)
chains2 = pd.read_table(fname, usecols=[2,3], sep=r'\s+', skiprows=1, names=colname2).head(100)
chains3 = pd.read_table(fname, usecols=[4,5], sep=r'\s+', skiprows=1, names=colname3).head(100)
```

```

hdul = fits.open(fit_file)
data = hdul[-1].data
hdul.close()

interval = PercentileInterval(95)
scaled = interval(data)

# image dimensions
ny, nx = scaled.shape

# scaled physical coordinates
x_extent = ((0 - xtarg)/100, (nx - xtarg)/100)
y_extent = ((0 - ytarg)/100, (ny - ytarg)/100)

fig, ax = plt.subplots(1,2, figsize=(15, 8), constrained_layout=True)
im = ax[0].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=cm
ax[0].scatter(chains1['X1_CENTER'], chains1['Y1_CENTER'], s=12, color='yellow', label='Star 1')
ax[0].scatter(chains2['X2_CENTER'], chains2['Y2_CENTER'], s=12, color='cyan', label='Star 2')
ax[0].scatter(chains3['X3_CENTER'], chains3['Y3_CENTER'], s=12, color='hotpink', label='Star 3')
ax[0].legend(loc='lower right', markerscale=3)
cbar = fig.colorbar(im, ax=[ax[0], ax[1]], fraction=0.046, pad=0.04)
cbar.set_label('Residuals')
ax[0].set_title("MCMC Chains (3star-fit)")
ax[0].set_xlabel("X (scaled pixels)")
ax[0].set_ylabel("Y (scaled pixels)")
ax[0].set_xlim(-10, 10)

ax[1].imshow(scaled, origin='lower', extent=(x_extent[0], x_extent[1], y_extent[0], y_extent[1]), cmap=cus
ax[1].set_xlim(-30, -10)
ax[1].set_title("PSF Residuals")
ax[1].set_xticklabels([])
ax[1].set_yticklabels([])
plt.show()

```

In []: